Chapter 7: Pickup-and-Delivery Problems for People Transportation

Vehicle Routing: Problems, Methods, and Applications, 2nd ed. (2014)

Karl F. Doerner Juan José Salazar-González

Slides prepared for self-study, 2026

Chapter Overview

Introduction – Motivation and Context I

Modern societies operate transportation services that require high efficiency and comfort, but also individual attention: transportation on demand, management of costs, and flexibility. These requirements apply to the entire transportation chain and involve different participants — drivers, customers, and the companies or public agencies providing the service.
The class of **Pickup-and-Delivery Problems for People (PDP-People)** covers this broad set of needs. Unlike goods-delivery variants where the payload is passive, *people* can have preferences, time constraints driven by their schedules, and a maximum acceptable in-vehicle ride time dictated by comfort and contractual service levels.

Introduction – Motivation and Context II

What distinguishes PDP-People from goods-based VRP?

- Each customer must be **picked up** at one location and **dropped off** at another, forming an origin–destination pair called a *request*.
- Customers have **time windows** at both the pickup and the delivery location.
- There is a **maximum ride time** L_i — a passenger cannot remain in the vehicle longer than a prescribed limit.
- The optimization must balance **operational cost** (fleet size, distance, driver time) against **service quality** (punctuality, ride-time comfort).

Introduction – Motivation and Context III

Real-world applications of PDP-People include:

- **Dial-a-Ride services**: door-to-door transit for elderly and disabled passengers who cannot use fixed public transport.
- **Medical transport**: non-emergency patient transfer to hospitals, clinics, dialysis centres.
- **School bus routing**: collecting students from home or bus stops and delivering them to one or more schools.
- **Demand responsive transit (DRT)**: semi-flexible bus systems that adapt routes in response to online booking.
- **Car pooling / ride-sharing**: sharing a vehicle for commuting when passengers have overlapping origin–destination corridors.

The chapter focuses on the **Dial-a-Ride Problem (DARP)** as the central, most extensively studied model, before surveying related variants. The key takeaway from the introduction is that people-transportation problems share the same combinatorial structure as goods-VRP but add a human-comfort dimension that makes modelling and solving them considerably harder.

Dial-a-Ride Problems – Definition and Context I

The **Dial-a-Ride Problem (DARP)** models a transportation system serving a fleet of vehicles that carry passengers from their origins to their destinations. The name originates from the 1970s service model where passengers *dial in* a ride request to a dispatch centre, which then assigns a vehicle and a time slot.

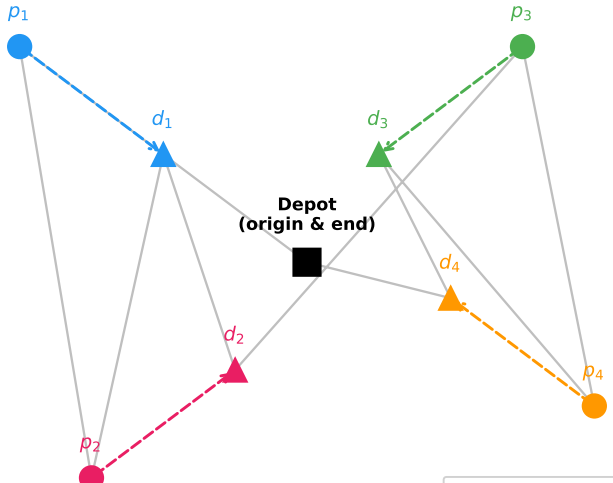
Informal Definition (DARP)

Given a set of transportation requests — each specifying an origin, a destination, a load (number of passengers), and time-window preferences — and a homogeneous fleet of vehicles starting and ending at a common depot, find a set of vehicle routes that serves all requests while respecting capacity, time-window, and maximum ride-time constraints, minimising total cost (typically a weighted combination of total route duration and total travel distance).

Dial-a-Ride Problems – Definition and Context II

The structural difference from VRPTW. In a Vehicle Routing Problem with Time Windows (VRPTW), each customer node is visited once and the order of visits is unrestricted. In the DARP, requests come in *pairs*: a pickup node p_i and a delivery node d_i. The vehicle must visit p_i before d_i on the same route. Additionally, the elapsed time between visiting p_i and d_i — the passenger's in-vehicle ride time — must not exceed L_i. This *precedence-plus-ride-time* coupling dramatically increases the difficulty of the problem.

Dial-a-Ride Problems – Definition and Context III



Mathematical Formulation of DARP I

We follow the formulation presented by Cordeau and Laporte (2003) and Parragh, Doerner, and Hartl (2008). The model is the most widely used basis for both exact and heuristic approaches.

Sets and notation. Let n be the number of transportation requests, numbered $1, \ldots, n$. Each request i has:

- a **pickup node** $i \in P = \{1, \ldots, n\}$, and
- a **delivery node** $n + i \in D = \{n + 1, \ldots, 2n\}$.

The depot is represented by two copies: node 0 (vehicle start) and node $2n + 1$ (vehicle end). The complete node set is $V = \{0\} \cup P \cup D \cup \{2n + 1\}$.

We have a homogeneous fleet of m vehicles, each with capacity Q. The travel time between nodes i and j is t_{ij}; the service time at node i is s_i; and node i has a **time window** $[e_i, l_i]$ meaning service must start no earlier than e_i and no later than l_i. The maximum ride time for request i is L_i, and the maximum route duration is T.

Mathematical Formulation of DARP II

Decision variables.

- $x_{ijk} \in \{0, 1\}$: equals 1 if vehicle k travels directly from node i to node j.
- B_{ik}: the time at which vehicle k begins service at node i.
- Q_{ik}: the load of vehicle k when it *leaves* node i.
- L_{ik}: the ride time of customer i on vehicle k.

Mathematical Formulation of DARP III

DARP: The Whole Problem (7.3.1)

Objective — minimise total route duration:

$$\min \sum_{k=1}^{m} \sum_{i \in V} \sum_{j \in V} c_{ij}\, x_{ijk}$$

where c_{ij} is the cost (typically travel time or distance) of arc (i, j). The sum counts every arc traversed by any vehicle.

Constraints:

$$\sum_{k=1}^{m} \sum_{j \in V} x_{ijk} = 1 \qquad \forall i \in P \qquad \text{(C1)}$$

$$\sum_{j \in V} x_{0jk} = 1 \qquad \forall k \qquad \text{(C2)}$$

$$\sum_{i \in V} x_{ihk} - \sum_{j \in V} x_{hjk} = 0 \qquad \forall h \in P \cup D,\ \forall k \qquad \text{(C3)}$$

Exact Methods: Branch-and-Cut (Cordeau 2006) I

The DARP is NP-hard. Exact methods guarantee an optimal solution but are computationally feasible only for moderate-sized instances (up to roughly $n \approx 8$ requests per vehicle, $m \leq 3$ vehicles in reasonable time). The state-of-the-art exact approach is the **branch-and-cut** algorithm of Cordeau (2006), which combines:

1. Linear programming (LP) relaxation at each tree node.
2. Valid inequalities (*cutting planes*) to tighten the LP bound.
3. Branching on fractional arc variables.

Branch-and-Cut Framework

1. Solve LP relaxation of the DARP formulation.
2. If the LP solution is integer-feasible: update incumbent if better, backtrack.
3. Else, separate violated valid inequalities. If found, add them and go to step 1. Otherwise, branch on a fractional variable $x^*_{ijk} \in (0, 1)$, creating two child nodes.
4. Prune a node if its LP lower bound exceeds the current incumbent.

Exact Methods: Branch-and-Cut (Cordeau 2006) II

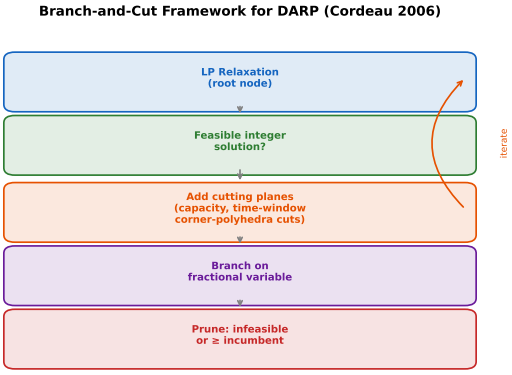


Figure: Schematic of the branch-and-cut framework for DARP. The iterate arrow (right side) shows that cutting planes can be added in multiple rounds before a branching decision is made, tightening the LP relaxation.

Exact Methods: Branch-and-Cut (Cordeau 2006) III

Key classes of valid inequalities used in DARP branch-and-cut:

- **Capacity cuts**: ensure that any subset S of requests cannot be served with fewer vehicles than required by their combined load. These are analogous to capacity cuts in CVRP branch-and-cut.
- **Time-window strengthening**: propagating the time-window tightness through the sequence of visits eliminates fractional solutions that would violate temporal feasibility.
- **Corner-polyhedra cuts**: when a scheduling subproblem has a fractional solution, cutting planes from the corner polyhedron of the scheduling polytope can be added to strengthen the formulation.
- **Precedence cuts**: the requirement that p_i precedes d_i on the same route generates logical implications on arc variables that can be turned into linear inequalities.

Exact Methods: Branch-and-Cut (Cordeau 2006) IV

Cordeau (2006) showed that on the standard benchmark instances (up to $n = 6$ requests and $m = 3$ vehicles), the branch-and-cut finds provably optimal solutions within reasonable CPU time. Larger instances require heuristic methods.

Key Insight

The principal bottleneck for exact DARP methods is the interaction between capacity and scheduling constraints: every branching decision that fixes a route arc immediately propagates time and load information, creating a highly constrained sub-MIP. Cutting planes that tighten both the capacity and the scheduling polytopes simultaneously are therefore the most valuable.

Heuristic Methods: Insertion and Improvement I

Because exact methods are limited to small instances, practitioners rely on **heuristic** approaches that trade optimality guarantees for speed. Two classical families are relevant:

1. Insertion heuristics. These build routes from scratch by iteratively selecting the "cheapest feasible" insertion position for each unrouted request. The algorithm terminates when all requests have been inserted or it is declared infeasible to do so (requiring a new vehicle).

Generic Insertion Heuristic for DARP

1. Initialise: routes $R_1, \ldots, R_m$ each containing only the depot.
2. Select unrouted request i (e.g., earliest time window first).
3. For each route R_k and each pair of positions (π_p, π_d) (where $\pi_p < \pi_d$) in R_k: compute the *insertion cost* $\Delta c(i, \pi_p, \pi_d, R_k)$ = increase in route cost from inserting p_i at position π_p and d_i at π_d.
4. Check feasibility: time windows, capacity, ride time L_i all satisfied after insertion.
5. Insert request i into the cheapest feasible slot found. If none exists, open a new vehicle route.
6. Repeat from step 2 until all requests are assigned.

Heuristic Methods: Insertion and Improvement II

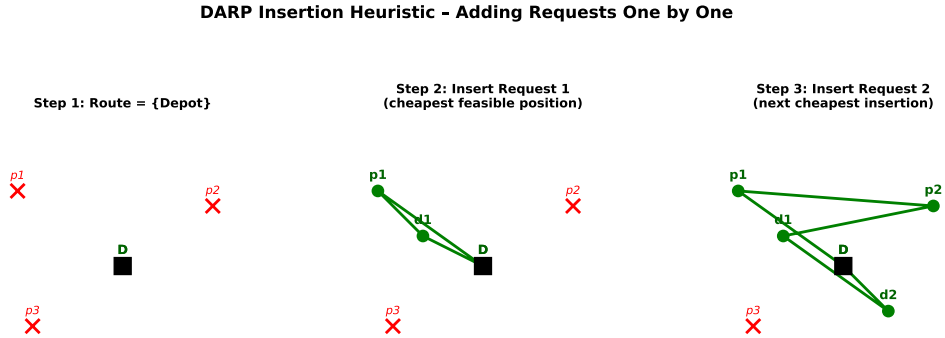


Figure: Three successive steps of a DARP insertion heuristic. At each step, the next request (red crosses) is inserted at the cheapest feasible position in the growing route (green). The route gradually grows as more requests are accommodated.

Worked Example: Insertion Cost Calculation

Suppose the current route visits nodes $[D, p_1, d_1, D]$ in order, with travel times $t_{D,p_1} = 5$, $t_{p_1,d_1} = 3$, $t_{d_1,D} = 4$. The current cost is $5 + 3 + 4 = 12$.

Now we want to insert request 2 with pickup p_2 and delivery d_2. Consider inserting p_2 between p_1 and d_1, and d_2 between d_1 and D. The new route is $[D, p_1, p_2, d_1, d_2, D]$. New cost: $t_{D,p_1} + t_{p_1,p_2} + t_{p_2,d_1} + t_{d_1,d_2} + t_{d_2,D}$. If $t_{p_1,p_2} = 2$, $t_{p_2,d_1} = 4$, $t_{d_1,d_2} = 3$, $t_{d_2,D} = 5$, new cost $= 5 + 2 + 4 + 3 + 5 = 19$. Insertion cost $= 19 - 12 = 7$.

Before accepting, we check: (a) capacity does not exceed Q at any position; (b) time windows $[e_{p_2}, l_{p_2}]$ and $[e_{d_2}, l_{d_2}]$ are satisfied; (c) ride time $B_{d_2,k} - (B_{p_2,k} + s_{p_2}) \leq L_2$. Only if all three checks pass is this insertion accepted.

Heuristic Methods: Insertion and Improvement IV

2. Improvement heuristics: 2-opt and 3-opt for DARP. After obtaining an initial solution, local search operators attempt to reduce cost by reconfiguring parts of the routes. The classical *2-opt* move for TSP reverses a segment of a route. For DARP, this operator must be modified to preserve:

- Pickup-before-delivery precedence for every request affected.
- Time-window feasibility at every node.
- Maximum ride-time feasibility for every passenger.

This makes feasibility checking more expensive than in TSP; efficient data structures that propagate the earliest and latest feasible service times ("forward time slack" and "backward time slack") allow checking in $O(1)$ per move after $O(n)$ preprocessing.

Key Insight

The high density of DARP constraints means that the feasible neighbourhood is much smaller than in VRPTW. Consequently, heuristics that can escape infeasibility — such as allowing temporary constraint violations with a penalty — tend to outperform strictly feasibility-preserving methods.

Metaheuristics for DARP I

Metaheuristics are higher-level strategies that guide local search to escape local optima. Three families dominate the DARP literature.

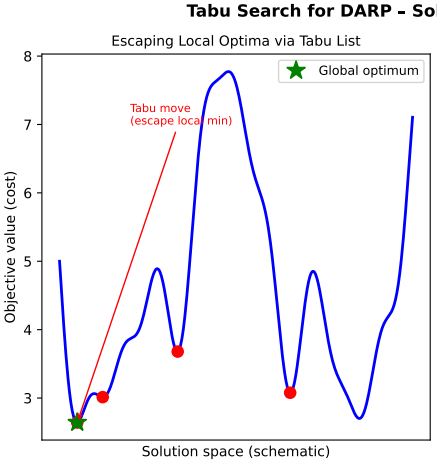
Tabu Search (TS). The idea behind tabu search is to allow moves to temporarily *worse* solutions, preventing cycling by declaring recently visited solutions or recently used moves *tabu* (forbidden) for a number of iterations. For DARP, Cordeau and Laporte (2003) developed an influential TS algorithm where the "move" is a *request relocation*: a single request i is removed from its current route and re-inserted into the same or another route at the cheapest feasible position.

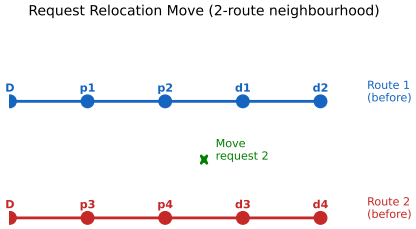
Metaheuristics for DARP II

Tabu Search for DARP (Cordeau & Laporte 2003)

1. Generate initial solution s (e.g., via insertion heuristic).
2. Repeat until stopping criterion (iteration limit or time limit):
   - 2a. For each request i and each candidate insertion position (π_p, π_d) on all routes: compute move cost $\delta(i, \pi_p, \pi_d)$.
   - 2b. Determine the best non-tabu move (or a tabu move if it yields a new best solution — *aspiration criterion*).
   - 2c. Execute the move; update tabu list (mark the reverse move tabu for θ iterations).
   - 2d. If current solution is better than best known, update incumbent s^*.
3. Return s^*.

Metaheuristics for DARP III





Metaheuristics for DARP IV

Figure: Left: schematic objective landscape for DARP. Tabu search allows uphill moves (red circles = local optima) to reach the global optimum (green star). Right: a "request relocation" neighbourhood move — request 2 is moved from Route 1 to Route 2.

Metaheuristics for DARP V

An important implementation detail for DARP tabu search: penalty functions. Cordeau and Laporte (2003) allow constraint violations during the search, penalising excess ride time, excess load, and violated time windows in the objective:

$$f(s) = \text{cost}(s) + \alpha \cdot \text{ExcessRideTime}(s) + \beta \cdot \text{ExcessLoad}(s) + \gamma \cdot \text{TimeWindowViolation}(s)$$

The penalty parameters $\alpha, \beta, \gamma > 0$ are adjusted *dynamically* during the search: if the current solution is feasible, they increase to push the search back towards feasibility; if infeasible, they decrease to allow more exploration. This self-regulating mechanism has been shown to be highly effective in practice.

Large Neighbourhood Search (LNS) / Adaptive LNS (ALNS). The idea of LNS, due to Shaw (1998), is to *destroy* a large portion of the current solution (removing q requests from their routes) and then *repair* it (re-inserting the removed requests in the cheapest feasible way). Because q can be large (e.g., $q = 5$–30), LNS explores a much larger neighbourhood than single-request relocation moves.

Metaheuristics for DARP VII

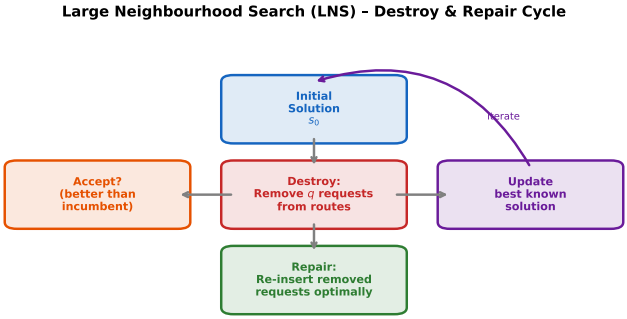


Figure: The Destroy-Repair cycle of Large Neighbourhood Search. After repair, the new solution is accepted if it improves the incumbent; the best solution is tracked throughout and returned at termination.

Metaheuristics for DARP VIII

Ropke and Pisinger (2006) extended LNS to **Adaptive LNS (ALNS)** by maintaining multiple destroy and repair operators and adaptively selecting them based on their historical performance. The operator selection follows a roulette-wheel mechanism: each operator's weight is updated after each iteration based on whether it contributed to a new best solution. For DARP, the destroy operators include:

- **Random removal**: select q requests uniformly at random.
- **Worst removal**: remove the q requests contributing most to the objective (most expensive rides).
- **Related removal** (Shaw 1998): remove q requests that are *similar* in time, location, or load — intuition is that similar requests can "swap" routes effectively.
- **Shaw removal with ride-time focus**: remove requests with the longest actual ride time first, as these are most likely causing constraint violations.

Worked Example: Related Removal

Consider 5 requests with the following direct travel time from a reference request r: $t(r,1) = 2$, $t(r,2) = 8$, $t(r,3) = 3$, $t(r,4) = 1$, $t(r,5) = 6$. Their time-window similarity scores (smaller = more similar): $\sigma_1 = 4$, $\sigma_2 = 1$, $\sigma_3 = 5$, $\sigma_4 = 2$, $\sigma_5 = 3$.

A relatedness measure combining distance and time: $\rho(r,i) = \lambda \cdot t(r,i) + (1-\lambda) \cdot \sigma_i$, with $\lambda = 0.5$: $\rho(r,1) = 3$, $\rho(r,2) = 4.5$, $\rho(r,3) = 4$, $\rho(r,4) = 1.5$, $\rho(r,5) = 4.5$.

Sorting by ρ: requests $4, 1, 3, 2, 5$. If $q = 3$, we remove requests $\{r, 4, 1\}$ — the three most related ones — and re-insert them using a greedy insertion repair.

Metaheuristics for DARP X

Population-based methods and hybrid approaches. Beyond single- solution metaheuristics, the literature includes:

- **Genetic algorithms**: encode DARP routes as chromosomes and apply crossover and mutation operators. The challenge is that standard GA crossover (e.g., order-crossover) can violate pickup-before-delivery precedence, so specialised crossover operators are needed.
- **Simulated annealing**: the "annealing" analogy comes from controlled cooling of metals — a slow temperature decrease makes the search accept worse solutions with decreasing probability, allowing global exploration early and exploitation later.
- **Branch-and-price with heuristic pricing**: in column- generation approaches, each feasible DARP route (column) is generated by solving a resource-constrained shortest-path problem. Heuristic pricing replaces exact labelling when the latter is too slow.

Metaheuristics for DARP XI

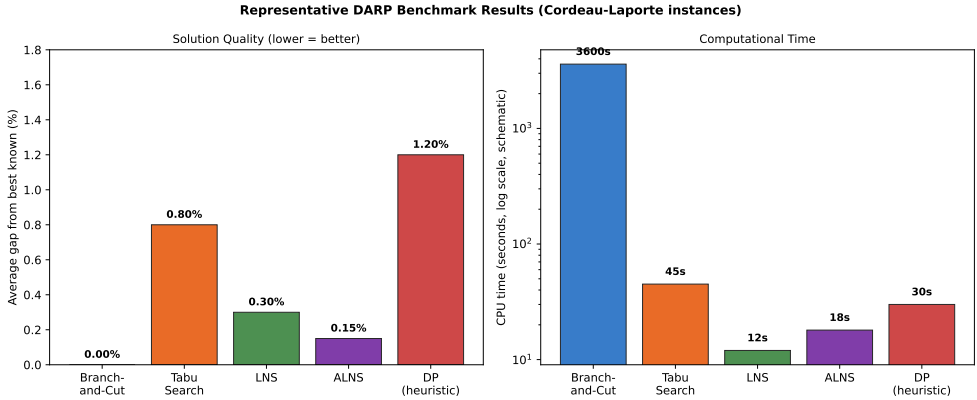
Key Insight

Across the DARP literature, metaheuristics — particularly tabu search and LNS/ALNS — consistently outperform pure constructive heuristics by 1–5% in solution quality, and they scale to hundreds of requests while exact methods stall at tens. The penalty-based infeasibility tolerance is the single most impactful design choice for tabu search.

Benchmark Instances and Computational Results I

The standard benchmark for DARP research is the set of instances introduced by Cordeau and Laporte (2003), denoted a (easy) and b (harder), with varying numbers of requests ($n = 24, 36, 48, 72, 96$) and vehicle fleet sizes ($m = 2, 3, \ldots, 13$). Each instance specifies a maximum route duration T, maximum ride time L, and vehicle capacity Q.

Benchmark Instances and Computational Results II



Benchmark Instances and Computational Results III

Figure: Schematic comparison of DARP solution methods on standard benchmark instances. Left: average gap from best known solution (lower is better). Right: typical CPU time (log scale). Branch-and-Cut achieves the best quality but requires orders of magnitude more time than metaheuristics for large instances.

Benchmark Instances and Computational Results IV

Observations from benchmark results:

- Branch-and-cut (Cordeau 2006) solves instances up to $n \approx 8$ requests per vehicle optimally; beyond this, computing time becomes prohibitive.
- The tabu search of Cordeau and Laporte (2003) finds solutions within 1% of optimum on small instances and remains the reference for medium instances ($n \leq 96$).
- ALNS (Ropke and Pisinger 2006) achieves the best solution quality on large instances and runs faster per iteration than tabu search due to simpler neighbourhood evaluation.
- Dynamic programming heuristics (Lim et al. 2010) handle single-vehicle instances optimally but do not scale to multi-vehicle problems.

Worked Example: Reading a Cordeau-Laporte Instance

Instance a2-16 has $n = 16$ requests, $m = 2$ vehicles, $Q = 6$ passengers, $T = 480$ minutes, $L = 90$ minutes. The 16 requests are distributed in a $[-10, 10]^2$ Euclidean plane. A solver would seek two routes each visiting at most 16 pickup/delivery pairs, each respecting time windows on each node, each with route duration ≤ 480, and each passenger's ride time ≤ 90.

Python Implementation: DARP Insertion Heuristic

Listing 1: DARP insertion heuristic skeleton (Python 3)

```python
import numpy as np
from itertools import permutations

def darp_insertion_heuristic(requests, n_vehicles, capacity, max_ride_time, travel_time):
    """
    Simple DARP insertion heuristic.
    requests: list of dict with keys: pickup, delivery, load, e_p, l_p, e_d, l_d
    travel_time: 2D numpy array, travel_time[i][j] = time from i to j
    Returns: list of routes (each route = ordered list of node indices)
    """
    DEPOT_START, DEPOT_END = 0, 2*len(requests) + 1
    routes = [[DEPOT_START, DEPOT_END] for _ in range(n_vehicles)]
    service_times = [0.0] * (2*len(requests) + 2)  # B[i] for each node

    def check_feasibility(route, svc):
        """Check time window + ride time + capacity along a route."""
        load, t = 0, 0.0
        for idx, node in enumerate(route):
            if idx > 0:
                t = max(t + travel_time[route[idx-1], node], 0)
            if node == DEPOT_START or node == DEPOT_END:
                continue
            req_idx = (node - 1) % len(requests)
            req = requests[req_idx]
            is_pickup = (node - 1) < len(requests)
```

School Bus Routing Problem (SBRP) I

The **School Bus Routing Problem (SBRP)** is one of the most practically important PDP-People variants. It differs from DARP in several structural ways:

SBRP vs. DARP: Key Structural Differences

DARP	**SBRP**
Door-to-door service	Pickup at designated *bus stops*
One depot	One or more *schools* as destinations
Heterogeneous time	Fixed bell times (hard deadlines)
Max ride time L_i	Walk distance $\leq W_{\max}$ from home to stop
General fleet	School buses (homogeneous or heterogeneous)

School Bus Routing Problem (SBRP) II

The SBRP consists of two interrelated subproblems:

1. **Bus stop selection**: choose which candidate stops to activate, ensuring every student lives within $W_{\max}$ walking distance of at least one active stop.
2. **Route generation and assignment**: design bus routes that visit the selected stops, pick up students, and arrive at the school on time.

These two subproblems are often solved sequentially (first stop selection, then routing), but joint optimisation yields better solutions.

School Bus Routing Problem (SBRP) III

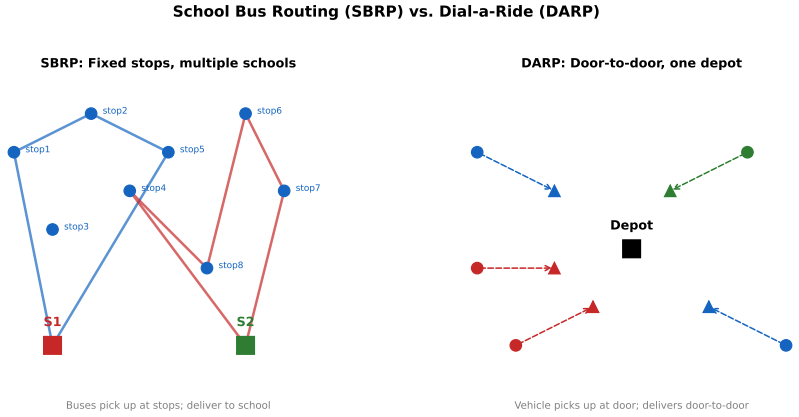


Figure: Left: SBRP with fixed stops (circles) and two schools (squares). Buses collect students from stops and deliver to schools on two separate routes. Right: DARP with door-to-door service from a single depot.

School Bus Routing Problem (SBRP) IV

Solution approaches for SBRP. Park and Kim (2010) provide a thorough survey. The most successful methods include:

- **GIS-based clustering**: students are first grouped into clusters by geographic proximity; each cluster is served by one bus. A Branch-and-Cut algorithm then optimises the routing within each cluster.
- **Mixed-integer programming**: for small instances, the joint stop-selection and routing problem can be solved exactly using CPLEX or Gurobi with the formulation of Bowman and Casto (1981).
- **Metaheuristics**: genetic algorithms (with cluster-first/ route-second encoding), tabu search, and simulated annealing are all applied in the literature; differences of 5–20% in fleet size are reported between methods.

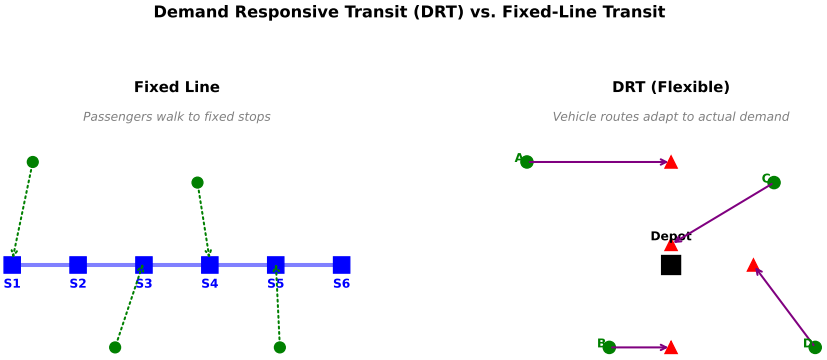
Key Insight

SBRP has a *spatial pre-processing* step — stop selection — that has no analogue in DARP. The walk-distance constraint couples the stop selection to the demographic distribution of students, making it a bi-level problem in general.

Demand Responsive Transit (DRT) I

Demand Responsive Transit (DRT) occupies a middle ground between fixed-line public transport (rigid timetables and fixed routes) and pure dial-a-ride (fully flexible door-to-door). DRT services operate on partially flexible routes that can deviate from a baseline path to serve on-demand pickup and delivery requests.

Demand Responsive Transit (DRT) II



Demand Responsive Transit (DRT) III

Figure: Left: fixed-line transit — passengers walk to fixed stops (S1–S6) along a predetermined route. Right: DRT — the vehicle route adapts to the actual distribution of demand, picking up passengers closer to their origins and delivering them nearer their destinations.

Demand Responsive Transit (DRT) IV

The "deviation" model for DRT. In the deviation model, each vehicle follows a *backbone route* that connects major activity centres. It is allowed to deviate by at most Δ minutes from this backbone to serve on-demand passengers. This constraint limits the service area and simplifies scheduling, at the cost of some flexibility.

DRT Deviation Constraint

Let $\tau_{\text{backbone}}(v)$ be the scheduled arrival time of vehicle v at backbone node b_j, and $\tau_{\text{actual}}(v)$ the actual arrival. The constraint is:

$$|\tau_{\text{actual}}(v, b_j) - \tau_{\text{backbone}}(v, b_j)| \leq \Delta \quad \forall j, \forall v$$

This ensures that connecting passengers relying on backbone timetables are not disappointed by large deviations.

Demand Responsive Transit (DRT) V

Online vs. offline DRT planning. When requests are known in advance (*offline*), DRT reduces to a DARP with additional backbone-deviation constraints, solvable by standard DARP metaheuristics. When requests arrive in real time (*online*), the system must re-route vehicles while in service, using:

- **Rolling horizon**: periodically re-solve the DARP for the next time horizon of H minutes using current vehicle positions as new depot locations.
- **Event-driven insertion**: each new request triggers a fast insertion into the current schedule; if infeasible, the request is rejected or deferred.

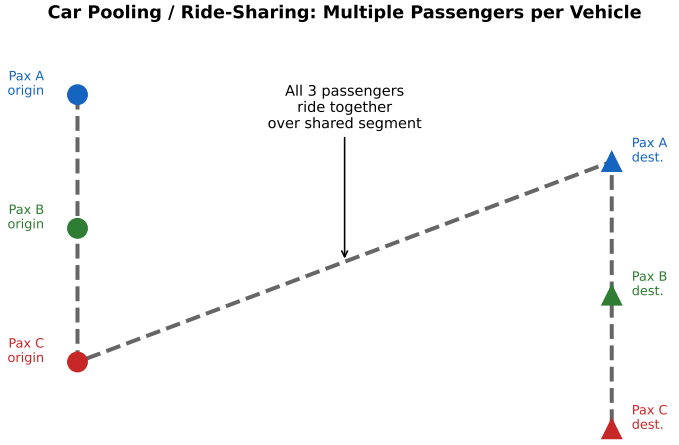
Key Insight

The key distinction of DRT from pure DARP is that DRT must respect a backbone timetable, limiting route deviations. This additional constraint can actually simplify the scheduling problem in some cases (fewer possible insertion positions) while adding complexity in others (backbone synchronisation requirements).

Car Pooling and Ride Sharing I

Car pooling (or *carpooling*) refers to the problem of assigning passengers with overlapping origin–destination paths to shared vehicles, reducing the total number of vehicle trips. Unlike DARP, the vehicles in car pooling are typically private cars whose owners (*drivers*) are also passengers with their own origin and destination.

Car Pooling and Ride Sharing II



Patient Transportation and Healthcare Logistics I

An important application domain for DARP is **patient transportation**: non-emergency medical transport of patients to hospitals, clinics, and dialysis centres. This domain has specific features that distinguish it from generic DARP:

- **Hard appointment times**: patients must arrive at the clinic no later than their appointment time (hard constraint, not just a preference).
- **Heterogeneous vehicles**: the fleet may include standard cars, wheelchair-accessible vans, and stretcher ambulances; patients can only use compatible vehicles.
- **Return trips**: after treatment, patients need a return trip home, often after an uncertain treatment duration (stochastic service time).
- **Multiple compatibility constraints**: a patient on dialysis may require a vehicle with medical equipment; pairing incompatible patients in the same vehicle is not permitted.

Patient Transportation and Healthcare Logistics II

Parragh, Doerner, and Hartl (2008) provide an extensive taxonomy of PDP variants including patient transport, classifying problems by whether pickup and delivery involve the same or different origins/destinations ("many-to-many" vs. "many-to-one" vs. "one-to-many" structures).

Key Insight

Patient transportation is a *many-to-many* PDP where vehicle heterogeneity, hard time constraints, and compatibility requirements all interact. Approaches must handle "soft" service time uncertainty (treatment duration) which pushes the problem into the realm of stochastic or robust optimisation.

Conclusions I

This chapter has surveyed the class of **Pickup-and-Delivery Problems for People Transportation**, with a focus on the Dial-a-Ride Problem (DARP) as the canonical model.

What we have covered:

1. **Problem definition and formulation**: the DARP is a mixed-integer program with binary arc variables, continuous service time variables, and constraints for capacity, time windows, pickup-before-delivery precedence, maximum ride time, and route duration.
2. **Exact methods**: branch-and-cut using LP relaxation, valid inequalities (capacity cuts, time-window strengthening, corner-polyhedra cuts), and branching on fractional arc variables. Solvable optimally for instances up to $n \approx 8$ per vehicle.
3. **Heuristics**: insertion heuristics build solutions sequentially; 2-opt and related improvement heuristics refine them using forward/backward time-slack data structures.
4. **Metaheuristics**: tabu search with penalty-based infeasibility tolerance (Cordeau & Laporte 2003), LNS and ALNS with destroy-repair operators (Ropke & Pisinger 2006).
5. **Variants**: school bus routing (SBRP), demand responsive transit (DRT), car pooling, and patient transportation each add domain-specific constraints to the DARP skeleton.

Conclusions II

Future research directions:

- **Dynamic and real-time DARP**: with smartphones and GPS, ride requests arrive online. Re-optimisation methods that integrate real-time data with look-ahead planning (e.g., stochastic DP, approximate DP, rollout algorithms) are an active research frontier.
- **Electric vehicle DARP**: charging time constraints and range limitations add a new resource dimension (energy) to the standard DARP model, requiring range-aware route planning.
- **Autonomous vehicle dial-a-ride**: without a human driver, vehicles can operate 24/7 and can be repositioned when idle. New objective functions and fairness criteria are needed.
- **Multi-modal DARP**: integrating ride-hailing with public transit (walk-to-bus, ride, walk-to-door) into a single optimisation model.
- **Data-driven and machine learning approaches**: using historical demand patterns to predict request distributions and pre-position vehicles; learning good neighbourhood operators for metaheuristics from data.
- **Equity and fairness**: ensuring that service quality (ride time, waiting time) is fairly distributed across different demographic groups, not just minimising aggregate cost.

Conclusions III

Chapter Key Takeaway

The DARP captures the essential difficulty of people-transportation logistics: the human comfort constraint (maximum ride time) couples the scheduling of pickups and deliveries in a way that makes both exact and heuristic solution fundamentally more complex than goods-VRP. State-of-the-art metaheuristics — tabu search and ALNS — are the practical tools of choice, with branch-and-cut providing an exact benchmark for small instances.

Key References I

The following references are the most frequently cited in this chapter. They span the foundational formulations, exact algorithms, and the most influential metaheuristics.

[1] J.-F. Cordeau and G. Laporte.
A tabu search heuristic for the static multi-vehicle dial-a-ride problem.
Transportation Research Part B, 37(6):579–594, 2003.

[2] J.-F. Cordeau.
A branch-and-cut algorithm for the dial-a-ride problem.
Operations Research, 54(3):573–586, 2006.

[3] S. Ropke and D. Pisinger.
An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows.
Transportation Science, 40(4):455–472, 2006.

Key References II

[4] S.N. Parragh, K.F. Doerner, and R.F. Hartl.
A survey on pickup and delivery problems. Part II: Transportation between pickup and delivery locations.
Journal für Betriebswirtschaft, 58(2):81–117, 2008.

[5] P. Shaw.
Using constraint programming and local search methods to solve vehicle routing problems.
In *Principles and Practice of Constraint Programming*, LNCS 1520, pp. 417–431. Springer, 1998.

[6] J. Park and B.-I. Kim.
The school bus routing problem: A review.
European Journal of Operational Research, 202(2):311–319, 2010.

[7] A. Beaudry, G. Laporte, T. Melo, and S. Nickel.
Dynamic transportation of patients in hospitals.
OR Spectrum, 32(1):77–107, 2010.

Key References III

[8] U. Ritzinger, J. Puchinger, and R.F. Hartl.
A survey on dynamic and stochastic vehicle routing problems.
International Journal of Production Research, 54(1):215–231, 2016.

[9] H.N. Psaraftis.
Dynamic vehicle routing problems.
In B.L. Golden and A.A. Assad (eds.), *Vehicle Routing: Methods and Studies*, pp. 223–248. North-Holland, 1988.

[10] A. Colorni and G. Righini.
Modeling and optimizing dynamic dial-a-ride problems.
International Transactions in Operational Research, 8(2):155–166, 2001.

Chapter 7 at a Glance

Core Model (DARP)

- MIP with binary arc vars + continuous scheduling vars
- Constraints: capacity, time windows, precedence, max ride time, route duration
- NP-hard; benchmark: Cordeau-Laporte instances

Heuristics & Metaheuristics

- Insertion: cheapest feasible position
- Tabu Search: request relocation + penalty functions
- LNS/ALNS: destroy q requests, repair greedily

Exact: Branch-and-Cut

- LP relaxation + valid inequalities
- Capacity cuts, time-window strengthening, corner-polyhedra cuts
- Optimal for $n \lesssim 8$ requests/vehicle

Variants

- **SBRP**: stop selection + routing + walk distance
- **DRT**: flexible routes around backbone
- **Car pooling**: driver-passenger compatibility
- **Patient transport**: heterogeneous fleet, hard appointments

DARP = VRPTW + pickup-before-delivery + bounded ride time. These two extra constraints make it the most complex VRP variant for people transportation, yet the most societally important.